# Procesamiento de resultados y buenas prácticas

## 1.1 Introducción

Una vez establecida la conexión con la base de datos y ejecutadas las sentencias SQL mediante los objetos Statement, PreparedStatement o CallableStatement, el siguiente paso consiste en **procesar los resultados** obtenidos de dichas consultas.

En el contexto de JDBC, este procesamiento se realiza principalmente a través del objeto **ResultSet**, que actúa como una **tabla virtual** en memoria.
Cada fila del ResultSet representa un registro del conjunto de resultados devuelto por la base de datos, y cada columna corresponde a un campo de ese registro.

El programador puede desplazarse por las filas del resultado, acceder a los valores de las columnas por nombre o por posición, y en algunos casos incluso **modificar los registros directamente desde el propio ResultSet**.

Además, JDBC proporciona la clase **ResultSetMetaData**, que permite **obtener información estructural sobre las columnas** de la consulta (nombres, tipos de datos, número total de columnas, etc.).
Esta información resulta especialmente útil cuando se trabaja con consultas dinámicas o resultados desconocidos en tiempo de compilación.

En este apartado se abordarán los siguientes objetivos:

- Comprender la función del objeto ResultSet y su ciclo de vida.
- Identificar los distintos tipos de cursores y sus características.
- Aprender a recorrer y leer los datos devueltos por una consulta.
- Utilizar ResultSetMetaData para analizar la estructura de los resultados.
- Aplicar buenas prácticas para la gestión de recursos y el control de errores.

De esta forma, el alumno será capaz de **recuperar, mostrar y manipular la información devuelta por la base de datos** de manera eficiente y segura, completando así el flujo lógico del trabajo con JDBC:
**Conexión → Ejecución → Procesamiento → Cierre de recursos.**

## 1.2 El objeto ResultSet

Cuando se ejecuta una consulta SELECT desde Java mediante cualquiera de los objetos Statement, PreparedStatement o CallableStatement, el método executeQuery() devuelve un objeto de tipo **ResultSet**, que contiene todas las filas devueltas por la base de datos.

En esencia, un ResultSet funciona como una **tabla virtual en memoria**:

- Cada fila representa un registro de la consulta.
- Cada columna representa un campo o atributo de ese registro.
- Un **cursor** interno permite desplazarse por las filas una a una

### Métodos más comunes

A continuación, se enumeran los métodos más utilizados:

| Método | Descripción |
|---|---|
| next() | Mueve el cursor a la siguiente fila. Devuelve false si no hay más registros. |
| getInt("columna"), getString("columna"), getDouble("columna"), etc. | Devuelve el valor de la columna especificada. |
| beforeFirst() | Coloca el cursor antes de la primera fila. |
| afterLast() | Coloca el cursor después de la última fila. |
| first() / last() | Mueve el cursor a la primera o última fila (si el conjunto es desplazable). |
| absolute(int fila) | Mueve el cursor a la fila indicada (si el conjunto es desplazable). |
| getRow() | Devuelve el número de la fila actual. |
| close() | Cierra el conjunto de resultados y libera recursos. |

## Tipos de ResultSet

Por defecto, los ResultSet son **solo hacia adelante y de solo lectura**. No obstante, al crear el Statement se puede especificar el tipo de desplazamiento y el nivel de concurrencia:

```
Statement stmt = con.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE,
    ResultSet.CONCUR_UPDATABLE);
```

A continuación, se muestran los valores que pueden tomar estos dos parámetros:

| Tipo de desplazamiento | Descripción |
|---|---|
| TYPE_FORWARD_ONLY | Solo permite avanzar hacia delante (por defecto). |
| TYPE_SCROLL_INSENSITIVE | Permite moverse en cualquier dirección; no refleja cambios realizados en la BD mientras está abierto. |
| TYPE_SCROLL_SENSITIVE | Permite moverse en cualquier dirección y refleja cambios en tiempo real. |

| Nivel de concurrencia | Descripción |
|---|---|
| CONCUR_READ_ONLY | No permite modificar los datos (modo por defecto). |
| CONCUR_UPDATABLE | Permite actualizar o eliminar registros desde el propio ResultSet. |

## 1.3 El objeto ResultSetMetaData

Cuando se ejecuta una consulta mediante JDBC, a menudo conocemos las columnas que devolverá el resultado y accedemos a ellas por nombre o por posición.

Sin embargo, en algunos casos —como cuando se ejecutan consultas dinámicas, generadas por el usuario o desconocidas en tiempo de compilación— es necesario descubrir la estructura del conjunto de resultados en tiempo de ejecución.

Para ello, la API JDBC proporciona la clase ResultSetMetaData, que permite obtener información detallada sobre las columnas de un ResultSet.

El objeto ResultSetMetaData contiene los **metadatos** (datos sobre los datos) de una consulta: número de columnas, nombres, tipos SQL, longitudes máximas, posibilidad de nulos, etc.

Se obtiene a partir de un ResultSet ya existente mediante el método:

```
ResultSetMetaData meta = rs.getMetaData();
```

### Métodos más comunes

| Método | Descripción |
|---|---|
| getColumnCount() | Devuelve el número total de columnas del ResultSet. |
| getColumnName(int i) | Devuelve el nombre de la columna en la posición indicada. |
| getColumnTypeName(int i) | Devuelve el tipo SQL (por ejemplo: VARCHAR, INT, DOUBLE). |
| getColumnDisplaySize(int i) | Devuelve el tamaño máximo de la columna. |
| isNullable(int i) | Indica si la columna puede contener valores NULL. |
| isAutoIncrement(int i) | Indica si la columna es autoincremental. |
| getTableName(int i) | Devuelve el nombre de la tabla a la que pertenece la columna. |

IMPORTANTE: revisar los ejemplos del aula virtual (procesamiento-resultados.zip) donde se incluyen diferentes usos de ResultSet y ResultSetMetaData

## 1.4 Actualización, inserción y eliminación desde ResultSet

Además de permitir la lectura de los resultados, algunos objetos ResultSet pueden utilizarse para **modificar directamente los datos** en la base de datos. Esto incluye **actualizar**, **insertar** y **eliminar** registros, siempre que el conjunto de resultados se haya creado con la configuración adecuada.

**Requisitos para un ResultSet actualizable**

1. La consulta SQL debe referirse **a una sola tabla** (sin JOIN, GROUP BY ni funciones agregadas).
2. Debe incluir la **clave primaria** de la tabla.
3. El Statement o PreparedStatement debe crearse con los modos:

```
ResultSet.TYPE_SCROLL_INSENSITIVE
ResultSet.CONCUR_UPDATABLE
```

```
Statement stmt = con.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE,
    ResultSet.CONCUR_UPDATABLE);
```

**Métodos clave**

| Método | Descripción |
|---|---|
| updateXxx(columna, valor) | Asigna un nuevo valor al campo especificado. |
| updateRow() | Actualiza en la base de datos la fila actual. |
| moveToInsertRow() | Posiciona el cursor en una fila vacía para preparar una inserción. |
| insertRow() | Inserta la nueva fila en la tabla. |
| deleteRow() | Elimina la fila actual de la tabla. |

Ten en cuenta:

- **Utilizar PreparedStatement** para consultas con parámetros.
- **Evitar usar joins o funciones agregadas** si se necesita actualizar.
- **Cerrar correctamente los recursos** (ResultSet, Statement, Connection).
- **Comprobar si el registro existe** antes de insertar o eliminar.

- **Validar entradas** (por ejemplo, números de empleado o departamento).
- **No abusar de los ResultSet actualizables**: para grandes operaciones, es preferible usar SQL directo o transacciones.